

Photon flux

October 4, 2026

The flux is written in `src/photon_flux.cpp` and given as dN/dz_γ : number of photons per unit energy fraction, as in Eqs. (15)–(22) of arXiv:2606.05469.

1 Variables

The code works with the light-cone momentum of the photon q^+ , the photon energy ω and the energy fraction z_γ (the fraction of the beam energy carried by the photon, Petja's tables call it y). They are related by

$$\omega = \frac{q^+}{\sqrt{2}}, \quad z_\gamma = \frac{2\omega}{\sqrt{s_{NN}}} = \frac{\sqrt{2}q^+}{\sqrt{s_{NN}}}, \quad \text{so} \quad \frac{dN}{dq^+} = \frac{z_\gamma}{q^+} \frac{dN}{dz_\gamma}, \quad (1)$$

where $\sqrt{s_{NN}}$ is the collision energy per nucleon pair.

The code has three momentum fractions: z_γ (photon), $z = p^+/q^+$ (charm quark in the photon) and z_h (D^0 in the charm quark).

There are also three transverse impact parameters:

- b : between the centres of the two nuclei. $\Gamma_{AA}(b)$ and the neutron factors depend on it.
- r : from the centre of the emitting nucleus to the point where the photon is absorbed. The flux $f(z_\gamma, r)$ depends on it.
- b_d : dipole impact parameter, the position in the target nucleus where the dipole interacts (also called s in the flux, with $b = |\mathbf{r} - \mathbf{s}|$).

2 Different fluxes in this code

The flux is chosen with the environment variable `FLUX_MODEL` (the defaults are in `run_scripts/config.sh`):

- `EFF` (default): “effective” flux of arXiv:2404.09731 (Eq. 4).
- `PL`: flux of a point-like nucleus, at $r = b$.
- `WS`: flux for a Woods–Saxon charge distribution, at $r = b$.
- `TABLE`: a table $z_\gamma, f(z_\gamma)$ from the file `FLUX_FILE` (Section 5).

`PL` and `WS` depend on b , which is integrated by the code from $b = 0$ (small b is removed by Γ_{AA}). In `EFF` and `TABLE` this integral is already done, so the flux depends only on z_γ . Any other name stops the program with an error.

Two more settings change the flux:

- `CHANNEL`, the neutron class. The nuclei can break up and emit neutrons, which is measured in the zero degree calorimeters. The probability that a nucleus does not break up is $P_{\text{noEM}}(b) = \exp(-S/b^2)$ with $S = (17.4 \text{ fm})^2$. The flux is multiplied by P_{noEM} for `An0n`, by $P_{\text{noEM}}(1 - P_{\text{noEM}})$ for `Xn0n` and by 1 for any other name (`AnAn`). With `PL` or `WS`, the channel `PL(AnAn)` replaces Γ_{AA} by a cut $b > 14.2 \text{ fm}$.
- The probability $\Gamma_{AA}(b)$ that the two nuclei do not collide hadronically, read from `input/WS_photon_flux/Gamma_AA.dat`:

$$\Gamma_{AA}(b) = \exp\left[-\sigma_{NN} \int d^2b' T_A(b') T_A(|\mathbf{b} - \mathbf{b}'|)\right], \quad \int d^2b T_A(b) = A, \quad (2)$$

with $\sigma_{NN} = 92 \text{ mb}$. One makes a new file with `src/make_gamma_aa.py`. `GAMMA_AA_ONE=1` sets $\Gamma_{AA} = 1$ (only for `PL` and `WS`).

The Woods–Saxon parameters are $R = 6.49 \text{ fm}$ and $d = 0.54 \text{ fm}$, the same as in Petja's tables.

p+Pb collisions

TARGET=pA changes the collision from Pb+Pb to p+Pb (Section 6.1 of arXiv:2606.05469): the lead nucleus emits the photon and the target is a proton. It is used with `run_scripts/run_proton.sh`, and the results go to `output/pPb/`. Three things change:

- the collision energy $\sqrt{s_{NN}}$,
- $\Gamma_{AA}(b)$ is replaced by $\Gamma_{pA}(b) = \exp[-\sigma_{NN} T_A(b)]$, with σ_{NN} given by SIGMA_NN,
- there is no neutron factor, so CHANNEL is not used.

The flux is WS (default) or PL, integrated over b . PL uses a cut $b > 1.1 R_{PL}$ instead of Γ_{pA} . EFF is not available, because the target is not a nucleus. A TABLE flux can be used.

3 The formulas

For a point-like nucleus (arXiv:2606.05469, Eq. 18)

$$f^{PL}(z_\gamma, r) = \frac{\alpha Z^2}{\pi^2 z_\gamma} [z_\gamma m_N K_1(z_\gamma m_N r)]^2, \quad (3)$$

and for the Woods–Saxon charge distribution (Eq. 16), with the form factor F_A of Maximon and Schrack,

$$f^{WS}(z_\gamma, r) = \frac{\alpha Z^2}{\pi^2 z_\gamma} \left[\int_0^\infty dk_\perp \frac{k_\perp^2 F_A(k_\perp^2 + z_\gamma^2 m_N^2)}{k_\perp^2 + z_\gamma^2 m_N^2} J_1(k_\perp r) \right]^2. \quad (4)$$

The $k_\perp$ integral is tabulated up to $r = 14.2$ fm. Above it the two fluxes are the same and the code uses f^{PL} . PL and WS use Eqs. (15), (20), (22) of arXiv:2606.05469,

$$f(z_\gamma) = \int d^2b f(z_\gamma, r=b) \Gamma_{AA}(b) P_{\text{channel}}(b), \quad (5)$$

and EFF also integrates over the position in the target nucleus (arXiv:2404.09731, Eq. 4):

$$f^{\text{EFF}}(z_\gamma) = \frac{1}{A} \int d^2r f^{WS}(z_\gamma, r) \int d^2s T_A(s) \Gamma_{AA}(|\mathbf{r} - \mathbf{s}|) P_{\text{channel}}(|\mathbf{r} - \mathbf{s}|). \quad (6)$$

There is no cut on b in EFF: small b is removed by Γ_{AA} .

EFF does not depend on p_{D^0} , y or the dipole, so it is computed once (about 12 s) and saved in `input/flux_cache/EFF_<channel>.dat`. The next runs read this file. It is computed again by itself if the parameters or the Γ_{AA} table change. FLUX_NO_CACHE=1 turns this off.

4 How the flux gets into the calculation

In `src/photon_flux.cpp` there are two functions that give dN/dz_γ :

- Flux that depends on b (PL, WS): the function `photon_flux()` gives $2\pi b f(z_\gamma, r=b) \Gamma_{AA}(b) P_{\text{channel}}(b)$.
- Flux already integrated over b (EFF, TABLE): the function `integrated_photon_flux()` gives $f(z_\gamma)$.

The first one still has to be integrated over b ; the code does that as one of the variables of its Monte Carlo integration (VEGAS).

The cross section is computed in `src/cross_section_inclusive.cpp`. There the code asks which kind of flux it has and multiplies by the Jacobian of Eq. (1), in the same way for every flux:

```
double z_gamma = sqrt(2.0) * qp / par->ss;
double flux_z = flux_is_integrated(p) ? integrated_photon_flux(z_gamma)
                                     : photon_flux(b, z_gamma, p);
double flux = flux_z * z_gamma / qp;
```

This is the only place with the factor z_γ/q^+ . The flux functions have no factor of q^+ , so the flux can be changed without modifying the integrand.

Careful: dipole impact parameter b_d

The dipole amplitude depends on b_d , so it is given as Glauber samples, one file per value, `data/Pb/mve/glauber_mve_<b_d>` with $b_d = 0, 2, \dots, 32 \text{ GeV}^{-1}$ (17 files).

The fluxes EFF and TABLE only remove the integral over the flux b . The integral over b_d is still needed for every flux: `run_scripts/run_nucleus.sh` runs the program once for each Glauber file, and `plotting_scripts/cross_section.py` then computes

$$\frac{d\sigma}{dy d^2p_{D^0\perp}} = 2\pi \int db_d b_d \frac{d\sigma(b_d)}{dy d^2p_{D^0\perp}} \quad (7)$$

with Simpson's rule (`integrate_over_b`). So one run for a single Glauber file is not the cross section yet but one point of this b_d integral.

5 How to add another flux

5.1 A table in z_γ

No code has to be changed. Write a text file with two columns,

```
# comments start with '#'  
z_gamma    f(z_gamma) = dN/dz_gamma
```

with z_γ increasing and $f > 0$ (more columns are ignored), and run

```
FLUX_MODEL=TABLE FLUX_FILE=my_flux.dat ./run_scripts/run_nucleus.sh
```

- If the table has $z_\gamma dN/dz_\gamma$, divide by z_γ first. If it has $dN/d\omega$, multiply by $\sqrt{s_{NN}}/2$. If it has a logarithm, take the exponential first.
- The table is interpolated in $\ln z_\gamma$ and $\ln f$. Above its last z_γ the flux is zero. If it starts above the smallest z_γ of a point, the program stops with an error. The smallest value needed is $z_\gamma = \sqrt{p_{D^0}^2 + m_c^2} e^y / \sqrt{s_{NN}}$, at the smallest p_{D^0} and y of the run.
- The table is for one neutron class, so CHANNEL is then only a name in the output.

Petja's tables (arXiv:2404.09731) are in `input/Starlight_photon_flux/`. They contain $\ln(dN/dz_\gamma)$ from $z_\gamma = 10^{-4}$, for AnAn and An0n. The script `src/make_flux_table.py` converts them to the format above, in the files `flux_<PL|WS>_<AnAn|An0n>.dat`. Below 10^{-4} it uses $z_\gamma f = A \ln(c/z_\gamma)$, fixed by the first two points of the table.

5.2 A flux that depends on b

Add a part to `photon_flux()` that returns $2\pi b f(z_\gamma, b)$ for your flux. Add its name in `init_photon_flux()`, but not in `flux_is_integrated()`. The factors $\Gamma_{AA}(b)$ and $P_{\text{channel}}(b)$ are applied after it, as for PL and WS.